WinAgent — Local Windows Agent Architecture

Package `agent/winagent` · Python 3.13 · Prepared 28 Oct 2026 from the current source tree

> The agent is the only component that touches the user's computer. It runs as a normal (non-administrator) Windows user, connects **outward** to the WinAgent server over an authenticated WebSocket, and executes whitelisted operations inside folders and applications that are configured **locally**. It enforces its own safety checks independently of the server.

1. Design principles

- **Outbound only.** No listening port; the agent dials the server and reconnects with exponential back-off (1 s → 30 s).
- **Local authority.** Approved folders (roots) and allowed apps live in `%LOCALAPPDATA%\WinAgent\agent-config.json`. The server and dashboard can never widen them.
- **Defence in depth.** Sensitive operations need an approval hash even if the server says to run them; paths are sandboxed; executables are refused.
- **Never repeat an uncertain action.** Every operation is journaled before it starts; duplicates return the recorded result or `UNCERTAIN`.
- **One action at a time.** A single lock serialises operations so desktop/browser actions never interleave.
- **Stoppable without the server.** A local `EMERGENCY_STOP` file halts work even when offline.

2. Internal components

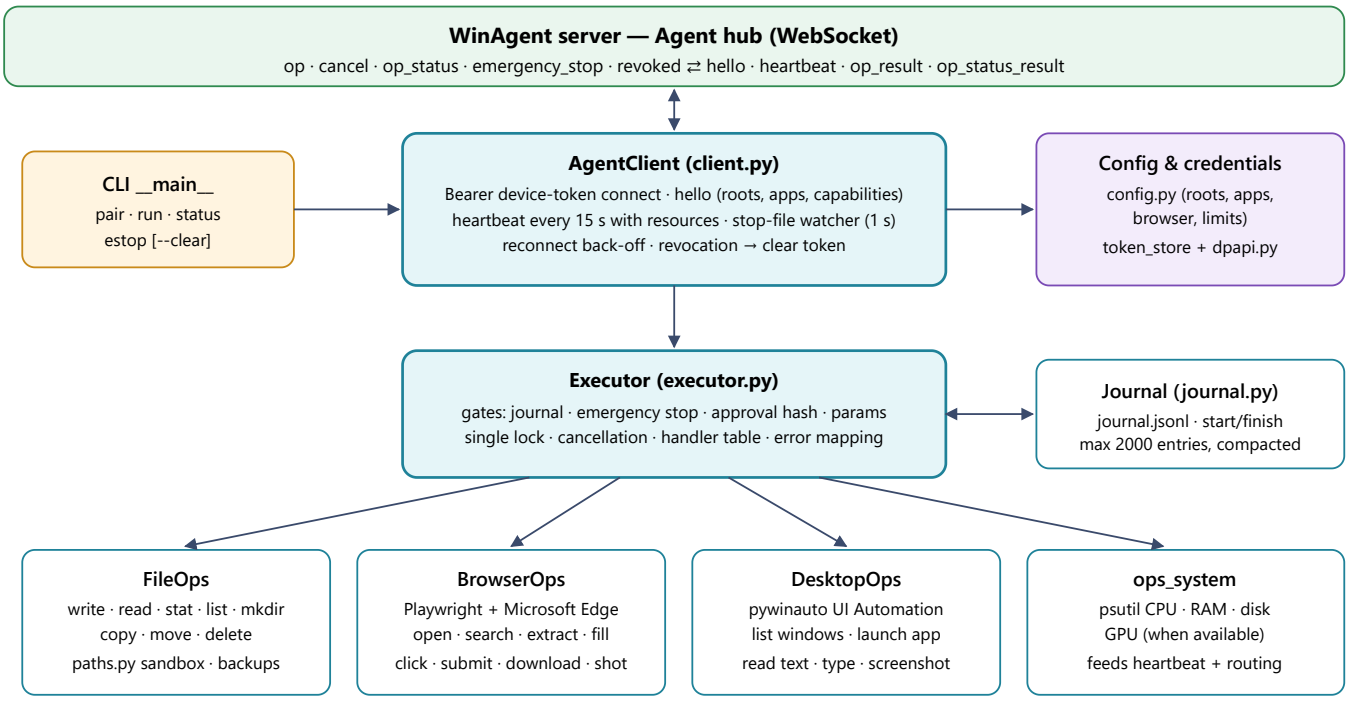
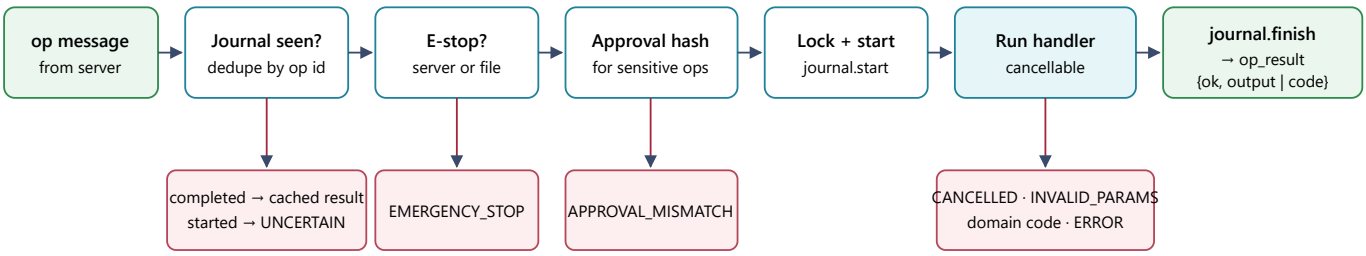


Figure 1 — Agent modules. Blocking operations run in worker threads (`asyncio.to_thread`); browser operations are async.

3. Operation execution pipeline



Sensitive operations requiring an approval hash on the agent: file.delete, browser.submit, browser.click, desktop.type_text.

Figure 2 — Every failure is reported back as a structured code; the agent process never crashes on an operation error.

4. Server ⇄ agent protocol

Message	Direction	Purpose
`hello`	agent → server	Version, hostname, OS, approved root names, allowed app names, capabilities (files, browser, uia, screenshots, ocr). The device receives no work until it has sent hello.
`heartbeat`	agent → server	Every 15 s: CPU/RAM/disk/GPU resources and local emergency-stop state.
`op`	server → agent	`operationId`, `operation`, `params`, `approvalHash`, `timeoutMs`, `taskId`.
`op_result`	agent → server	`{ok:true, output}` or `{ok:false, code, error}`.
`cancel`	server → agent	Cancels a running operation (task cancel / emergency stop).
`op_status` / `op_status_result`	both	Server asks the journal whether an operation completed before deciding to retry (completed / running / unknown).
`emergency_stop`	server → agent	Activate: cancel all running operations, close the browser, refuse new work. Clear: accept work again.
`revoked`	server → agent	Device revoked: the agent deletes its stored token and exits.

5. Operation catalogue

Area	Operation	Base risk	Notes
Files	`file.write`	LOW	Overwrite raises to MEDIUM; original backed up; returns SHA-256
	`file.read` · `file.stat` · `file.list`	LOW	Read limit 1 MB; list limit 500 entries
	`file.mkdir`	LOW	
	`file.copy`	LOW	Overwrite raises to MEDIUM; destination backed up
	`file.move`	MEDIUM	Source backed up first
	`file.delete`	HIGH	Approval required on server and agent; backed up
Browser	`browser.open` · `browser.search` · `browser.extract_text` · `browser.screenshot`	LOW	Search via DuckDuckGo; page text scanned for prompt injection
	`browser.fill`	MEDIUM	Sensitive fields (password, card, CVC, OTP) detected
	`browser.click`	MEDIUM	Agent still requires an approval hash
	`browser.submit`	HIGH	Approval required
	`browser.download`	MEDIUM	Size limit (default 50 MB); executable content refused; saved in an approved folder
Desktop	`desktop.list_windows` · `desktop.read_window_text` · `desktop.screenshot`	LOW	UI Automation via pywinauto
	`desktop.launch_app`	LOW	Only allow-listed apps from local config
	`desktop.type_text`	HIGH	Typed in 20-character chunks; stops mid-way on cancel or E-stop
System	`system.resources`	LOW	CPU, memory, disk, GPU

6. Safety mechanisms on the agent

Mechanism	Behaviour
Path sandbox (`paths.py`)	Every path is a root name + relative path; resolved path must stay inside the root. Rejects traversal, absolute paths, reserved device names (CON, PRN, NUL, COM1…) and executable extensions (.exe, .msi, .ps1, .bat, .dll, .lnk, …).
Backups	Overwrite, move and delete copy the original to `backups\<timestamp>\` first, so changes are reversible.
Human-verification detection	reCAPTCHA, hCaptcha, Cloudflare challenges, OTP inputs and "verify you are human / 2FA" text return `HUMAN_VERIFICATION_REQUIRED`; the server blocks the task for the user. Never bypassed.
Prompt-injection detection	Extracted web text is scanned for instructions such as "ignore previous instructions" or "send your password" and flagged in the result.
Privilege	`run` refuses to start elevated unless `--allow-elevated` is passed.
Emergency stop	Three triggers: server message, local stop file (CLI `estop` or Electron tray), checked every second and before every operation.
Credentials	Device token encrypted with Windows DPAPI (user scope, extra entropy); cleared on revocation.

7. Lifecycle

1. **Pair:** admin creates a one-time code in *Agent devices*; `python -m winagent pair --code <CODE>` posts it to `/api/agents/pair` and stores the returned device token with DPAPI.
2. **Run:** `python -m winagent run` loads config (creating defaults on first run), connects, sends hello, starts heartbeat and stop-file watcher, then handles messages.
3. **Reconnect:** on connection loss, retry with back-off up to 30 s. HTTP 401 or a `revoked` message ends the process (re-pair required).
4. **Status / stop:** `winagent status` prints config and pairing state; `winagent estop` / `--clear` toggles the local stop file.

8. Local storage — `%LOCALAPPDATA%\WinAgent\`

Item	Contents
`agent-config.json`	Server URL, device name, approved roots (default `workspace` = Documents\WinAgentWorkspace, `test` = …\test), allowed apps, browser channel (msedge), headless flag, download limit, heartbeat interval
`device-token.dpapi`	DPAPI-encrypted device id + token
`journal.jsonl`	Operation start/finish records for idempotency
`evidence\`	Browser and desktop screenshots per task
`backups\`	Originals preserved before overwrite / move / delete
`EMERGENCY_STOP`	Present only while the local emergency stop is active